

Graphs & Trees

Complete Competitive Programming Reference

Every Algorithm • Key Properties • CF Problems • Rating 800 – 2300

Ch	Topic	Algorithms Covered	Rating
1	Graph Representation & Traversal	Adjacency list/matrix, BFS, DFS, cycle detection	800–1400
2	Shortest Paths	Dijkstra, Bellman-Ford, SPFA, Floyd-Warshall, 0-1 BFS	1200–1900
3	Minimum Spanning Tree	Kruskal, Prim, Boruvka, properties	1200–2000
4	Trees — Core Algorithms	DFS order, diameter, centroid, LCA, rerooting DP	1200–2100
5	Advanced Tree Techniques	HLD, centroid decomposition, small-to-large, DSU on tree	2000–2300
6	Graph Special Structures	Bipartite, SCC, bridges, articulation, topological sort	1500–2200
7	Flows & Matching	Max flow, min cut, bipartite matching, min cost flow	1800–2300
8	Graph DP	DP on DAG, tree DP, DP on paths, broken profile	1500–2300
9	Master Algorithm Selector	Quick reference — what to use when	All

Chapter 1 — Graph Representation & Traversal

Rating: 800 – 1400

Graph Representations

Type	Space	Add Edge	Check Edge	Best for
Adjacency List	$O(V+E)$	$O(1)$	$O(\text{deg})$	Sparse graphs — standard in CP
Adjacency Matrix	$O(V^2)$	$O(1)$	$O(1)$	Dense graphs, small V (≤ 1000)
Edge List	$O(E)$	$O(1)$	$O(E)$	Kruskal MST, sorting edges

Standard Adjacency List Template

```
const int MAXN = 200005;

vector<int> adj[MAXN];           // unweighted
vector<pair<int,int>> wadj[MAXN]; // weighted: {neighbour, weight}

// Add undirected edge:
adj[u].push_back(v); adj[v].push_back(u);

// Add directed edge:
adj[u].push_back(v);
```

BFS — Breadth First Search

TRIGGER: Shortest path in unweighted graph, level-by-level processing

- Visits nodes level by level — guarantees shortest path in unweighted graphs
- Time: $O(V+E)$ | Space: $O(V)$
- Use `dist[]` array initialised to -1; set `dist[src]=0` before pushing to queue
- Multi-source BFS: push ALL sources with `dist=0` at the start

```
vector<int> bfs(int src, int n) {
    vector<int> dist(n+1, -1);
    queue<int> q;
    dist[src] = 0; q.push(src);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
    return dist;
}
```

DFS — Depth First Search

TRIGGER: Connected components, cycle detection, topological sort, tree properties

- Explores as deep as possible before backtracking
- Time: $O(V+E)$ | Space: $O(V)$ for recursion stack
- `visited[]` prevents revisiting; `tin[]/tout[]` for entry/exit times

- Use iterative DFS for large graphs to avoid stack overflow

```
vector<int> tin(MAXN), tout(MAXN), color(MAXN, 0);

int timer = 0;

void dfs(int u, int par = -1) {
    color[u] = 1;    // grey: currently visiting
    tin[u] = timer++;
    for (int v : adj[u]) {
        if (v == par) continue;        // skip parent in undirected
        if (color[v] == 1) { /* back edge -> cycle */ }
        if (color[v] == 0) dfs(v, u);
    }
    color[u] = 2;    // black: fully processed
    tout[u] = timer++;
}
```

Cycle Detection

Graph type	Method	Key observation
Undirected	DFS — back edge to non-parent means cycle	Track parent; if visited[v] and v != parent -> cycle
Directed	DFS with 3-colour marking	Grey node during DFS = back edge = cycle
Any	Union-Find — adding edge between same component	If find(u)==find(v) before union -> cycle

Practice Problems

Problem	Source	Rating	Key Insight
Counting Rooms	CSES	1000	DFS/BFS — count connected components
Labyrinth	CSES	1100	BFS shortest path, reconstruct the path
Building Roads	CSES	1100	Count components, connect with minimum edges
Message Route	CSES	1200	BFS shortest path with path reconstruction
Monsters	CSES	1200	Multi-source BFS from all monster positions
Round Trip	CSES	1300	Find a cycle — DFS back edge detection
Course Schedule	CSES	1300	Detect cycle in directed graph (topological sort)
Reachability Queries	CSES	1500	DFS in/out times for ancestor queries

Chapter 2 — Shortest Paths

Rating: 1200 – 1900

Algorithm Comparison

Algorithm	Complexity	Weights	Negative cycles	Use when
BFS	$O(V+E)$	Unweighted only	N/A	All edges weight 1
0-1 BFS	$O(V+E)$	0 or 1 only	No	Edges cost 0 or 1
Dijkstra	$O((V+E) \log V)$	Non-negative	No	Standard weighted SSSP
Bellman-Ford	$O(VE)$	Any	Detects	Negative edges, small graph
SPFA	$O(VE)$ worst	Any	Detects	Negative edges, faster in practice
Floyd-Warshall	$O(V^3)$	Any	Detects	All-pairs, $V \leq 500$

Dijkstra — Standard Template

- Uses min-heap priority queue; greedily processes nearest unvisited node
- FAILS with negative edges — use Bellman-Ford instead
- `dist[]` initialised to INF; `dist[src]=0`
- Lazy deletion: skip if `dist[u] < current popped distance`

```
typedef pair<long long,int> pli;
const long long INF = 1e18;

vector<long long> dijkstra(int src, int n) {
    vector<long long> dist(n+1, INF);
    priority_queue<pli, vector<pli>, greater<pli>> pq;
    dist[src] = 0; pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue; // lazy deletion
        for (auto [v, w] : wadj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

0-1 BFS — for edges with weight 0 or 1

- Replace queue with deque; push weight-0 edges to front, weight-1 to back
- $O(V+E)$ — faster than Dijkstra when all edges are 0 or 1
- Common pattern: choosing to pay cost 1 or 0 based on some condition

```
vector<int> bfs01(int src, int n) {
    vector<int> dist(n+1, INT_MAX);
    deque<int> dq;
    dist[src] = 0; dq.push_back(src);
```

```

while (!dq.empty()) {
    int u = dq.front(); dq.pop_front();
    for (auto [v, w] : wadj[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            if (w == 0) dq.push_front(v);
            else      dq.push_back(v);
        }
    }
}
return dist;
}

```

Bellman-Ford & Negative Cycle Detection

```

// Returns true if negative cycle reachable from src
bool bellman_ford(int src, int n, vector<tuple<int,int,int>>& edges) {
    vector<long long> dist(n+1, INF); dist[src] = 0;
    for (int i = 0; i < n-1; i++) // relax n-1 times
        for (auto [u,v,w] : edges)
            if (dist[u] != INF && dist[u]+w < dist[v])
                dist[v] = dist[u]+w;
    // nth iteration: if any relaxation -> negative cycle
    for (auto [u,v,w] : edges)
        if (dist[u] != INF && dist[u]+w < dist[v]) return true;
    return false;
}

```

Floyd-Warshall — All Pairs Shortest Path

```

long long dist[505][505];
// Initialise: dist[i][i]=0, dist[i][j]=edge weight or INF
for (int k=1;k<=n;k++)
    for (int i=1;i<=n;i++)
        for (int j=1;j<=n;j++)
            dist[i][j] = min(dist[i][j], dist[i][k]+dist[k][j]);
// Negative cycle: if dist[i][i] < 0 for any i

```

Practice Problems

Problem	Source	Rating	Key Insight
Shortest Routes I	CSES	1100	Dijkstra SSSP — standard template
Shortest Routes II	CSES	1200	Floyd-Warshall APSP
High Score	CSES	1500	Bellman-Ford — maximize path (negate weights)
Flight Discount	CSES	1500	Dijkstra — use at most one discounted edge
Teleporters Path	CSES	1600	Eulerian path in directed graph
Investigation	CSES	1700	Dijkstra + count paths + min/max edges
Elevator Rides	CSES	1900	Bitmask DP on shortest path — groups
Road Reparation	CF 59E	1900	Dijkstra variant with special cost model

Problem	Source	Rating	Key Insight
Minimum Cost Path	CF 1765F	2000	0-1 BFS on modified state graph
Frog 1 & 2	AtCoder	1200	DP on DAG — classic shortest path formulation

Chapter 3 — Minimum Spanning Tree

Rating: 1200 – 2000

TRIGGER: Connect all nodes with minimum total edge weight

Key MST Properties

- A spanning tree of n nodes has exactly $n-1$ edges
- Cut property: the minimum weight edge crossing any cut belongs to some MST
- Cycle property: the maximum weight edge in any cycle does NOT belong to any MST
- If all edge weights are distinct, the MST is unique
- MST total weight is unique even if the tree itself is not (with equal weights)
- Second MST: replace one MST edge with a non-tree edge — $O(E \log E)$

Kruskal's Algorithm — $O(E \log E)$

- Sort edges by weight; greedily add edges that don't create a cycle
- Use Union-Find (DSU) for cycle detection in $O(\alpha(n))$ per operation
- Best for sparse graphs — standard choice in CP

```
struct DSU {
    vector<int> par, rnk;
    DSU(int n): par(n+1), rnk(n+1,0) { iota(par.begin(), par.end(), 0); }
    int find(int x){ return par[x]==x ? x : par[x]=find(par[x]); }
    bool unite(int a, int b){
        a=find(a); b=find(b);
        if(a==b) return false;
        if(rnk[a]<rnk[b]) swap(a,b);
        par[b]=a; if(rnk[a]==rnk[b]) rnk[a]++;
        return true;
    }
};

long long kruskal(int n, vector<tuple<int,int,int>>& edges) {
    sort(edges.begin(), edges.end()); // sort by weight
    DSU dsu(n);
    long long total = 0; int cnt = 0;
    for (auto [w,u,v] : edges) {
        if (dsu.unite(u,v)) { total += w; cnt++; }
        if (cnt == n-1) break;
    }
    return (cnt == n-1) ? total : -1; // -1 if not connected
}
```

Prim's Algorithm — $O((V+E) \log V)$

- Grows the tree one vertex at a time using a priority queue
- Better for dense graphs; similar to Dijkstra in implementation
- `key[]` = minimum edge weight to add each vertex to the growing tree

```
long long prim(int n) {
    vector<long long> key(n+1, INF);
    vector<bool> inMST(n+1, false);
```

```
priority_queue<pli,vector<pli>,greater<pli>> pq;
key[1]=0; pq.push({0,1});
long long total=0;
while(!pq.empty()){
    auto [d,u]=pq.top(); pq.pop();
    if(inMST[u]) continue;
    inMST[u]=true; total+=d;
    for(auto [v,w]:wadj[u])
        if(!inMST[v] && w<key[v]){ key[v]=w; pq.push({w,v}); }
}
return total;
}
```

MST Variants & Applications

Problem type	Approach
Maximum spanning tree	Negate weights and run Kruskal, or sort descending
MST with some edges forced	Pre-add forced edges (union their endpoints), then run Kruskal on rest
MST with some edges forbidden	Remove forbidden edges, run Kruskal on remaining
Minimax path (min of max edge on path)	Answer = max edge on MST path between u and v
Number of MSTs	Matroid intersection — complex; count edges of same weight in each group
Steiner tree (subset of nodes)	Bitmask DP on tree — $O(3^k * V + 2^k * V * \log V)$

Practice Problems

Problem	Source	Rating	Key Insight
Building Roads	CSES	1100	MST — connect components
Road Construction	CSES	1200	Kruskal — track MST formation
Flight Routes Check	CSES	1300	Check if graph is strongly connected
Kruskal's Algorithm	CSES	1400	Direct Kruskal implementation
New Roads Queries	CSES	1800	MST + LCA — minimax path queries
Spanning Tree Queries	CSES	1800	MST properties — offline queries
Landscape Cities	CF 1468J	2000	MST + tree structure
Tourist	CF 1342F	2000	MST + tree DP on the MST
Xor-MST	CF 888G	2200	MST on XOR-weighted complete graph — trie

Chapter 4 — Trees — Core Algorithms

Rating: 1200 – 2100

Key Tree Properties

- A tree with n nodes has exactly $n-1$ edges and is connected
- There is exactly ONE path between any two nodes
- Removing any edge disconnects the tree
- Adding any edge creates exactly one cycle
- A node with degree 1 is a leaf
- Rooted tree: each node except root has exactly one parent

Tree Diameter

- Diameter = longest path between any two nodes
- Two-BFS method: BFS from any node, find farthest node u ; BFS from u , find farthest node v ; $\text{dist}(u,v)$ = diameter
- This works because the farthest node from any node is always a diameter endpoint
- For weighted trees, use BFS with weights (or DFS)

```
pair<int,int> farthest_bfs(int src, int n) {
    vector<int> dist(n+1,-1); queue<int> q;
    dist[src]=0; q.push(src);
    while(!q.empty()){
        int u=q.front(); q.pop();
        for(int v:adj[u]) if(dist[v]==-1){dist[v]=dist[u]+1;q.push(v);}
    }
    int far = max_element(dist.begin(),dist.end()) - dist.begin();
    return {far, dist[far]};
}

// Diameter:
auto [u, _] = farthest_bfs(1, n);
auto [v, diam] = farthest_bfs(u, n);
```

Tree Centroid

- Centroid: node whose removal leaves all components of size $\leq n/2$
- Every tree has 1 or 2 centroids
- Key property: depth from centroid is $O(\log n)$ after repeated centroid decomposition
- Used as root for divide-and-conquer on trees

```
int sz[MAXN]; bool removed[MAXN];
void calc_size(int u, int p){
    sz[u]=1;
    for(int v:adj[u]) if(v!=p && !removed[v]){ calc_size(v,u); sz[u]+=sz[v]; }
}
int find_centroid(int u, int p, int tree_sz){
    for(int v:adj[u])
        if(v!=p && !removed[v] && sz[v]>tree_sz/2) return find_centroid(v,u,tree_sz);
    return u;
}
```

LCA — Lowest Common Ancestor

TRIGGER: Path queries on tree, distance between nodes, ancestor checks

- Binary lifting: precompute $\text{anc}[v][k] = 2^k\text{-th ancestor of } v$ — $O(n \log n)$ build, $O(\log n)$ query
- Euler tour + sparse table: $O(n \log n)$ build, $O(1)$ query — faster for many queries
- $\text{dist}(u,v) = \text{depth}[u] + \text{depth}[v] - 2 \cdot \text{depth}[\text{LCA}(u,v)]$
- u is ancestor of v iff $\text{tin}[u] \leq \text{tin}[v]$ and $\text{tout}[v] \leq \text{tout}[u]$

```
const int LOG = 18;
int anc[MAXN][LOG], depth[MAXN];

void dfs_lca(int u, int p, int d){
    anc[u][0]=p; depth[u]=d;
    for(int k=1;k<LOG;k++) anc[u][k]=anc[anc[u][k-1]][k-1];
    for(int v:adj[u]) if(v!=p) dfs_lca(v,u,d+1);
}

int lca(int u, int v){
    if(depth[u]<depth[v]) swap(u,v);
    int diff=depth[u]-depth[v];
    for(int k=0;k<LOG;k++) if((diff>>k)&1) u=anc[u][k];
    if(u==v) return u;
    for(int k=LOG-1;k>=0;k--)
        if(anc[u][k]!=anc[v][k]){ u=anc[u][k]; v=anc[v][k]; }
    return anc[u][0];
}

int dist(int u, int v){ return depth[u]+depth[v]-2*depth[lca(u,v)]; }
```

Rerooting DP

- Compute $\text{dp}[v]$ = answer for the tree rooted at v , for all v
- Pass 1: root at node 1, compute $\text{dp_down}[v]$ (subtree answer)
- Pass 2: reroot — pass $\text{dp_up}[v]$ (contribution from rest of tree) downward
- Key: $\text{dp}[v] = \text{combine}(\text{dp_down}[v], \text{dp_up}[v])$
- Most common use: sum of distances from each node, max depth from each node

```
// Example: sum of distances from each node
long long down[MAXN], ans[MAXN];
int sz[MAXN];

void dfs1(int u,int p){ // compute subtree sizes and down-sum
    sz[u]=1; down[u]=0;
    for(int v:adj[u]) if(v!=p){
        dfs1(v,u); sz[u]+=sz[v]; down[u]+=down[v]+sz[v];
    }
}

void dfs2(int u,int p,long long up_sum){ // reroot
    ans[u] = down[u] + up_sum;
    for(int v:adj[u]) if(v!=p){
        long long new_up = (ans[u]-down[v]-sz[v]) + (n-sz[v]);
        dfs2(v,u,new_up);
    }
}
```

```

}
// Call: dfs1(1, -1); dfs2(1, -1, 0);

```

Practice Problems

Problem	Source	Rating	Key Insight
Tree Diameter	CSES	1100	Two-BFS diameter algorithm
Tree Distances I	CSES	1200	Rerooting DP — sum/max distances
Tree Distances II	CSES	1200	Rerooting — max distance from each node
Company Queries I	CSES	1400	Binary lifting — k-th ancestor
Company Queries II	CSES	1400	LCA with binary lifting
Distance Queries	CSES	1400	LCA — distance between nodes
Path Queries	CSES	1500	Euler tour + BIT — path sum queries
Subtree Queries	CSES	1500	Euler tour — subtree sum with updates
Tree Matching	CSES	1600	Greedy tree DP — maximum matching
Counting Paths	CSES	1700	Path counting — centroid or LCA
Kefa's Park	CF 580C	900	DFS on tree with constraint
New Year Tree	CF 750E	2000	HLD + segment tree on tree

Chapter 5 — Advanced Tree Techniques

Rating: 2000 – 2300

Heavy-Light Decomposition (HLD)

TRIGGER: Path queries / updates on tree in $O(\log^2 n)$

- Decompose tree into chains — each path crosses at most $O(\log n)$ chains
- Assign contiguous positions to each chain; run segment tree on this array
- Heavy child: the child with the largest subtree
- Any root-to-leaf path has at most $O(\log n)$ light edges
- Query path $u \rightarrow v$: split at LCA, query each chain segment
- Complexity: $O(n)$ build, $O(\log^2 n)$ per path query/update

```
int par[MAXN], dep[MAXN], sz[MAXN], heavy[MAXN], head[MAXN], pos[MAXN], cur_pos;

void dfs(int u, int p, int d){
    par[u]=p; dep[u]=d; sz[u]=1; heavy[u]=-1;
    int max_sz=0;
    for(int v:adj[u]) if(v!=p){
        dfs(v,u,d+1); sz[u]+=sz[v];
        if(sz[v]>max_sz){ max_sz=sz[v]; heavy[u]=v; }
    }
}

void decompose(int u, int h){
    head[u]=h; pos[u]=cur_pos++;
    if(heavy[u]!=-1) decompose(heavy[u],h); // continue chain
    for(int v:adj[u]) if(v!=par[u]&&v!=heavy[u]) decompose(v,v); // new chain
}

long long query_path(int u, int v){ // query path u->v using seg tree on pos[]
    long long res=0;
    while(head[u]!=head[v]){
        if(dep[head[u]]<dep[head[v]]) swap(u,v);
        res+=seg.query(pos[head[u]],pos[u]);
        u=par[head[u]];
    }
    if(dep[u]>dep[v]) swap(u,v);
    res+=seg.query(pos[u],pos[v]);
    return res;
}
```

Centroid Decomposition

TRIGGER: Count/optimize over ALL paths in tree — $O(n \log^2 n)$

- Recursively decompose tree using centroid; each centroid handles paths through it
- Every path passes through exactly one centroid in the decomposition
- Decomposition depth is $O(\log n)$ — key to efficiency
- Use case: count pairs at distance $\leq k$, path XOR queries, path sum problems

```
void solve_centroid(int c){
    // collect all distances from c in current component
```

```

vector<int> dists; dists.push_back(0);
for(int v:adj[c]) if(!removed[v]) get_dists(v,c,1,dists);
// process paths THROUGH c using dists
// e.g. count pairs at distance k:
// sort dists, use two-pointer

removed[c]=true;
for(int v:adj[c]) if(!removed[v]){
    calc_size(v,c);
    int nc=find_centroid(v,c,sz[v]);
    solve_centroid(nc);
}
}

```

Small-to-Large Merging (DSU on Tree)

- When merging sets at each node, always merge smaller into larger
- Each element is moved at most $O(\log n)$ times — total $O(n \log n)$
- Use for: count distinct values in subtree, most frequent element in subtree
- DSU on tree (Sack): process heavy child last, don't clear its data — $O(n \log n)$

```

// Small-to-large: merge sets up the tree
map<int,int> cnt[MAXN]; // cnt[u] = frequency map for subtree of u

void dfs(int u, int p){
    cnt[u][val[u]]++;
    for(int v:adj[u]) if(v!=p){
        dfs(v,u);
        if(cnt[v].size()>cnt[u].size()) swap(cnt[u],cnt[v]); // key: merge small into large
        for(auto [x,f]:cnt[v]) cnt[u][x]+=f;
    }
    ans[u] = cnt[u].size(); // or whatever query you need
}

```

Practice Problems

Problem	Source	Rating	Key Insight
Path Queries II	CSES	2000	HLD + segment tree — path max queries with updates
Distinct Colors	CSES	1700	Small-to-large or DSU on tree — distinct in subtree
Finding a Centroid	CSES	1600	Centroid finding algorithm
Counting Paths	CSES	1700	Centroid decomp — count paths of length k
New Year Tree	CF 750E	2000	HLD + seg tree with bitset for colors
Tree Queries	CF 486E	2100	HLD — path queries, tricky edge case
Arpa's letter-marked	CF 850C	1900	DSU on tree — count pairs in subtree
Lomsat gelral	CF 600E	2100	Small-to-large — most frequent in subtree
Ynoi2015 No.4	CF 906E	2300	Centroid decomp + segment tree
Colony	CF 1060F	2300	Centroid decomp — distance queries

Chapter 6 — Graph Special Structures

Rating: 1500 – 2200

Topological Sort

TRIGGER: Ordering of tasks with dependencies / processing DAG nodes

- Only possible on DAGs (Directed Acyclic Graphs)
- Kahn's algorithm (BFS): start with in-degree 0 nodes
- If processed nodes < n: cycle exists, no valid topological order

```
vector<int> topo_sort(int n){
    vector<int> indeg(n+1,0), order;
    for(int u=1;u<=n;u++) for(int v:adj[u]) indeg[v]++;
    queue<int> q;
    for(int u=1;u<=n;u++) if(indeg[u]==0) q.push(u);
    while(!q.empty()){
        int u=q.front(); q.pop(); order.push_back(u);
        for(int v:adj[u]) if(--indeg[v]==0) q.push(v);
    }
    return order; // size < n means cycle exists
}
```

Strongly Connected Components (SCC) — Kosaraju's

- SCC: maximal subgraph where every node is reachable from every other
- Condensation DAG: contract each SCC to one node — results in a DAG
- Kosaraju: DFS on original graph, DFS on reversed graph in reverse finish order
- Tarjan's: single DFS using low-link values — slightly more efficient
- Key property: in condensation DAG, source SCCs have no incoming edges

```
// Kosaraju's SCC
vector<int> order; vector<bool> vis(MAXN,false);
int comp[MAXN];
void dfs1(int u){ vis[u]=true; for(int v:adj[u]) if(!vis[v]) dfs1(v); order.push_back(u); }
void dfs2(int u, int c){ comp[u]=c; for(int v:radj[u]) if(!comp[v]) dfs2(v,c); }
int kosaraju(int n){
    for(int i=1;i<=n;i++) if(!vis[i]) dfs1(i);
    int num_scc=0;
    for(int i=n-1;i>=0;i--) if(!comp[order[i]]) dfs2(order[i],++num_scc);
    return num_scc;
}
```

Bridges & Articulation Points — Tarjan's

- Bridge: edge whose removal disconnects the graph
- Articulation point: vertex whose removal disconnects the graph
- Use low[] values: $low[v] = \min tin$ reachable from subtree of v via one back edge
- Bridge condition: $low[v] > tin[u]$ for edge (u,v) where v is child of u
- Articulation point: u is root with 2+ children, OR non-root with child v where $low[v] \geq tin[u]$

```
int tin[MAXN], low[MAXN], timer=0;
bool is_bridge[MAXNE], is_artic[MAXN];
```

```

void dfs(int u, int par_edge=-1){
    tin[u]=low[u]=timer++;
    int children=0;
    for(auto [v,eid]:adj[u]){ // adj stores {neighbour, edge_id}
        if(tin[v]==-1){
            children++;
            dfs(v,eid);
            low[u]=min(low[u],low[v]);
            if(low[v]>tin[u]) is_bridge[eid]=true;
            if(par_edge==-1&&children>1) is_artic[u]=true;
            if(par_edge!=-1&&low[v]>=tin[u]) is_artic[u]=true;
        } else if(eid!=par_edge) low[u]=min(low[u],tin[v]);
    }
}

```

Bipartite Graphs

- Bipartite: vertices can be 2-coloured such that no edge connects same colour
- A graph is bipartite iff it has no odd-length cycle
- Check with BFS/DFS colouring: if same-colour edge found -> not bipartite
- Max matching in bipartite graph: Hopcroft-Karp $O(E \cdot \sqrt{V})$
- Min vertex cover = Max matching (Konig's theorem)
- Max independent set = V - Max matching

Practice Problems

Problem	Source	Rating	Key Insight
Course Schedule	CSES	1300	Topological sort + cycle detection
Longest Flight Route	CSES	1400	Longest path in DAG — topo sort + DP
Game Routes	CSES	1400	Count paths in DAG — topo sort + DP
Planet Queries I/II	CSES	1500	Functional graph — binary lifting
Road Reparation	CSES	1500	Bridges in graph
Building Teams	CSES	1300	Bipartite check — 2-colouring
Giant Pizza	CSES	1700	2-SAT — SCC on implication graph
Checkposts	CF 427C	1700	SCC — minimum cost to cover all SCCs
Biconnected Components	CF 1045E	2100	Bridges + biconnected component tree
Two Sat	CSES	2000	2-SAT using SCC — classic application
Escape Room	CF 1348E	2200	SCC + flow on condensation DAG

Chapter 7 — Flows & Matching

Rating: 1800 – 2300

Key Theorems

- Max-Flow Min-Cut: maximum flow = minimum s-t cut capacity
- Konig's Theorem: in bipartite graph, max matching = min vertex cover
- Hall's Theorem: perfect matching exists iff for every subset S of left nodes, $|N(S)| \geq |S|$
- Max independent set = n - max matching (bipartite)
- Min path cover in DAG = n - max bipartite matching

Dinic's Algorithm — $O(V^2 * E)$

- Best general max-flow algorithm for CP
- On unit-capacity graphs: $O(E * \sqrt{V})$ — same as Hopcroft-Karp
- Build level graph with BFS, find blocking flows with DFS
- Blocking flow: DFS from source, advance if $\text{level}[v] = \text{level}[u] + 1$, retreat otherwise

```
struct Dinic {
    struct Edge { int to, rev; long long cap; };
    vector<Edge> g[MAXN]; int level[MAXN], iter[MAXN];

    void add_edge(int u, int v, long long cap){
        g[u].push_back({v, (int)g[v].size(), cap});
        g[v].push_back({u, (int)g[u].size()-1, 0}); // reverse edge cap=0
    }

    bool bfs(int s, int t, int n){
        fill(level, level+n+1, -1); queue<int> q;
        level[s]=0; q.push(s);
        while(!q.empty()){ int u=q.front(); q.pop();
            for(auto&e:g[u]) if(e.cap>0&&level[e.to]<0){level[e.to]=level[u]+1;q.push(e.to);}}
        return level[t]>=0;
    }

    long long dfs(int u, int t, long long f){
        if(u==t) return f;
        for(int&i=iter[u]; i<(int)g[u].size(); i++){
            Edge&e=g[u][i];
            if(e.cap>0&&level[v=e.to]==level[u]+1){
                long long d=dfs(v, t, min(f, e.cap));
                if(d>0){e.cap-=d; g[v][e.rev].cap+=d; return d;}
            }
        }
        return 0;
    }

    long long max_flow(int s, int t, int n){
        long long flow=0;
        while(bfs(s, t, n)){ fill(iter, iter+n+1, 0);
            long long d; while((d=dfs(s, t, INF))>0) flow+=d; }
        return flow;
    }
}
```



```
};
```

Bipartite Matching — Hungarian / Hopcroft-Karp

```
// Simple augmenting path matching — O(VE)

int match_l[MAXN], match_r[MAXN]; // matching for left/right nodes
bool vis[MAXN];

bool dfs(int u){
    for(int v:adj[u]){
        if(!vis[v]){ vis[v]=true;
            if(match_r[v]==-1||dfs(match_r[v])){ match_l[u]=v; match_r[v]=u; return true; }
        }
    }
    return false;
}

int max_matching(int n){
    fill(match_l,match_l+n+1,-1); fill(match_r,match_r+n+1,-1);

    int res=0;
    for(int u=1;u<=n;u++){ fill(vis,vis+n+1,false); if(dfs(u)) res++; }

    return res;
}
```

Flow Problem Modelling Patterns

Problem type	Model
Bipartite matching	Source -> left nodes (cap 1) -> right nodes (cap 1) -> sink
Min vertex cover (bipartite)	Max matching (Konig's). Nodes NOT in min cover = max independent set
Min path cover in DAG	$n - \text{max matching}$ where edge $u \rightarrow v$ in DAG becomes $\text{left}_u \rightarrow \text{right}_v$ in bipartite
Node capacity constraint	Split each node v into v_{in} and v_{out} with edge of given capacity
Multiple sources/sinks	Add super-source connected to all sources, super-sink connected to all sinks
Project selection (closure)	Source -> projects with profit cap, projects -> required items cap INF, items -> sink with cost

Practice Problems

Problem	Source	Rating	Key Insight
Max Flow	CSES	1700	Direct Dinic max flow implementation
Distinct Routes	CSES	1900	Flow — find edge-disjoint paths
School Dance	CSES	1700	Bipartite matching — boys and girls
Coin Grid	CSES	1700	Min vertex cover in bipartite = max matching
Download Speed	CSES	1500	Basic max flow / min cut
Police Chase	CSES	1600	Min cut = min edges to disconnect
Escape and Conquer	CF 1033E	2200	Flow with node capacities
Flea	CF 1239D	2100	Bipartite matching variant
Kingdom and its Cities	CF 613D	2200	Greedy on tree — related to matching
The Shortest Path	CF 59E	2100	Min cost flow — path decomposition

Chapter 8 — Graph DP

Rating: 1500 – 2300

DP on DAG

TRIGGER: Optimal path, count paths, longest path — in directed acyclic graph

- Process nodes in topological order — each node is processed after all its dependencies
- $dp[v]$ = optimal value achievable ending at v
- Longest path in DAG: $dp[v] = \max(dp[u] + w(u,v))$ for all predecessors u
- Count paths: $dp[v] = \text{sum of } dp[u] \text{ for all predecessors } u$
- Shortest path in DAG: Bellman-Ford on topological order in $O(V+E)$

```
// Longest path in DAG —  $O(V+E)$ 

vector<int> order = topo_sort(n); // topological order

vector<long long> dp(n+1, -INF); dp[src] = 0;

for (int u : order) {
    if (dp[u] == -INF) continue;
    for (auto [v,w] : wadj[u])
        dp[v] = max(dp[v], dp[u] + w);
}

// dp[dst] = length of longest path from src to dst
```

Tree DP — Classic Patterns

- Root the tree; $dp[v]$ depends only on $dp[\text{children of } v]$
- Process children before parent (post-order DFS)
- State often needs subtree size, depth, or a secondary dimension
- Common: $dp[v][0/1]$ = answer for subtree of v where v is not/is selected

```
// Maximum independent set on tree

int dp[MAXN][2]; // dp[v][0]=max set not including v, dp[v][1]=max including v

void dfs(int u, int p){
    dp[u][0]=0; dp[u][1]=1; // v not selected / v selected
    for(int v:adj[u]) if(v!=p){
        dfs(v,u);
        dp[u][0] += max(dp[v][0], dp[v][1]); // can take or not take child
        dp[u][1] += dp[v][0]; // must not take child if u is taken
    }
}

// Answer: max(dp[root][0], dp[root][1])
```

DP on Trees with Merging (Knapsack on Tree)

- $dp[v][j]$ = best answer using j nodes from subtree of v
- Merge children one by one: $O(\text{sz}[u] \cdot \text{sz}[v])$ per merge $\rightarrow O(n^2)$ total by subtree argument
- The key insight: total merging work is $O(n^2)$ because each pair of nodes is merged at their LCA exactly once

```
// Tree knapsack — select exactly k nodes to maximise value

vector<long long> dp[MAXN]; // dp[v][j] = best selecting j from subtree v

void dfs(int u, int p){
    dp[u].assign(sz[u]+1, -INF); dp[u][1] = val[u];
    for(int v:adj[u]) if(v!=p){
```

```

    dfs(v,u);

    // merge dp[u] and dp[v]

    vector<long long> ndp(dp[u].size()+dp[v].size()-1, -INF);

    for(int j=0;j<(int)dp[u].size();j++)
        for(int k=0;k<(int)dp[v].size();k++)
            if(dp[u][j]!=-INF && dp[v][k]!=-INF)
                ndp[j+k]=max(ndp[j+k], dp[u][j]+dp[v][k]);

    dp[u]=ndp;
}
}

```

Functional Graph DP

- Functional graph: each node has exactly one outgoing edge -> forms rho shape (path + cycle)
- Find cycles with DFS or Floyd's algorithm
- Binary lifting on functional graph: find where you end up after 2^k steps
- Classic: find period of a sequence, kth successor queries

Practice Problems

Problem	Source	Rating	Key Insight
Longest Flight Route	CSES	1400	Longest path in DAG — topo sort DP
Game Routes	CSES	1400	Count paths in DAG
Tree Matching	CSES	1600	Max matching on tree — DP[v][0/1]
Tree Coloring	CSES	1600	Count valid colorings — tree DP
Tree Knapsack	CSES	1900	Select k nodes — $O(n^2)$ tree knapsack
Planet Cycles	CSES	1600	Functional graph — find cycle lengths
Codeforces DP Round	CF 1551E	2000	Tree DP with two states
New Year Tree	CF 750E	2000	Tree DP + HLD + seg tree
Dish Owner	CF 1442E	2100	DP on DAG with complex state
Choosing Capital	CF 219D	1600	Rerooting DP — min edges to orient
Kingdom Diversity	CF 1089F	2200	Tree DP with merging — complex state

Chapter 9 — Master Algorithm Selector

See the problem structure. Know the algorithm. This is the most important page.

When you see...	Algorithm / Technique	Complexity	Rating
Shortest path, unweighted graph	BFS	$O(V+E)$	900
Connected components, cycle detection	DFS with colour marking	$O(V+E)$	900
Tasks with prerequisites / ordering	Topological Sort (Kahn's BFS)	$O(V+E)$	1200
All edges weight 0 or 1	0-1 BFS with deque	$O(V+E)$	1400
Weighted shortest path, no negative edges	Dijkstra with priority queue	$O((V+E) \log V)$	1200
Negative edge weights	Bellman-Ford or SPFA	$O(VE)$	1500
All pairs shortest path, $V \leq 500$	Floyd-Warshall	$O(V^3)$	1300
Detect negative cycle	Bellman-Ford nth iteration OR Floyd diag	$O(VE)$	1600
Minimum cost to connect all nodes	Kruskal (DSU) or Prim	$O(E \log E)$	1200
Minimax path between two nodes	Max edge on MST path (LCA + binary lifting)	$O(n \log n)$	1800
Tree diameter / farthest node	Two-BFS method	$O(n)$	1100
k-th ancestor / LCA queries	Binary lifting	$O(n \log n)$	1400
Many LCA queries fast	Euler tour + sparse table RMQ	$O(n \log n) / O(1)$	1600
Answer for each node as root	Rerooting DP (two-pass DFS)	$O(n)$	1600
Path sum/max/update queries on tree	Heavy-Light Decomposition + seg tree	$O(n \log^2 n)$	2000
Count/optimize over ALL tree paths	Centroid Decomposition	$O(n \log^2 n)$	2100
Distinct values / merge sets in subtree	Small-to-Large or DSU on tree	$O(n \log n)$	1900
Strongly connected subgraphs	Kosaraju or Tarjan SCC	$O(V+E)$	1700
Bridges / articulation points	Tarjan bridge finding (low values)	$O(V+E)$	1700
Is graph 2-colourable	BFS bipartite check	$O(V+E)$	1200
2-SAT (implication constraints)	SCC on implication graph	$O(V+E)$	2000
Maximum flow / minimum cut	Dinic's algorithm	$O(V^2 E)$	1700
Maximum bipartite matching	Hopcroft-Karp or augmenting paths	$O(E \sqrt{V})$	1700
Min vertex cover (bipartite)	Konig: = max matching	$O(E \sqrt{V})$	1800
Optimal assignment with costs	Min cost max flow (MCMF)	$O(VE \log V)$	2000
Optimal/count paths in DAG	DP in topological order	$O(V+E)$	1400
Select nodes from tree optimally	Tree DP with $dp[v][state]$	$O(n * states)$	1600
Select exactly k nodes from tree	Tree knapsack — $O(n^2)$ with merging	$O(n^2)$	1900
Functional graph (each node 1 outgoing edge)	Binary lifting for kth successor	$O(n \log n)$	1600

Every graph problem has a structural tell. Unweighted distances -> BFS. Negative weights -> Bellman-Ford. Connect all nodes cheaply -> Kruskal. Path queries on tree -> HLD. All paths through tree -> Centroid Decomp. Tasks with dependencies -> Topological sort. Routing maximum flow -> Dinic. Train yourself to see the structure first, then pick the algorithm. The algorithm choice is mechanical once you see the structure.